



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Centre de la Imatge i la Tecnologia Multimèdia

Lab Session 7: World state Replication



Previously, In Theory

- Object Replication: Which parts of the game represent state that is affected by multiple players?

ReplicationManagerServer

- Maps NetworkIDs to commands (examples: Update, Create, Destroy, Events, etc.). Use a Vector/List/Dictionary/Enumerable
- Writes a “replication packet”:
 - For Action-Object pair:
 - Get Object from Linking Context
 - Serialize relevant fields
- Sends package to client
- Can be done:
 - On Update()
 - Regular intervals

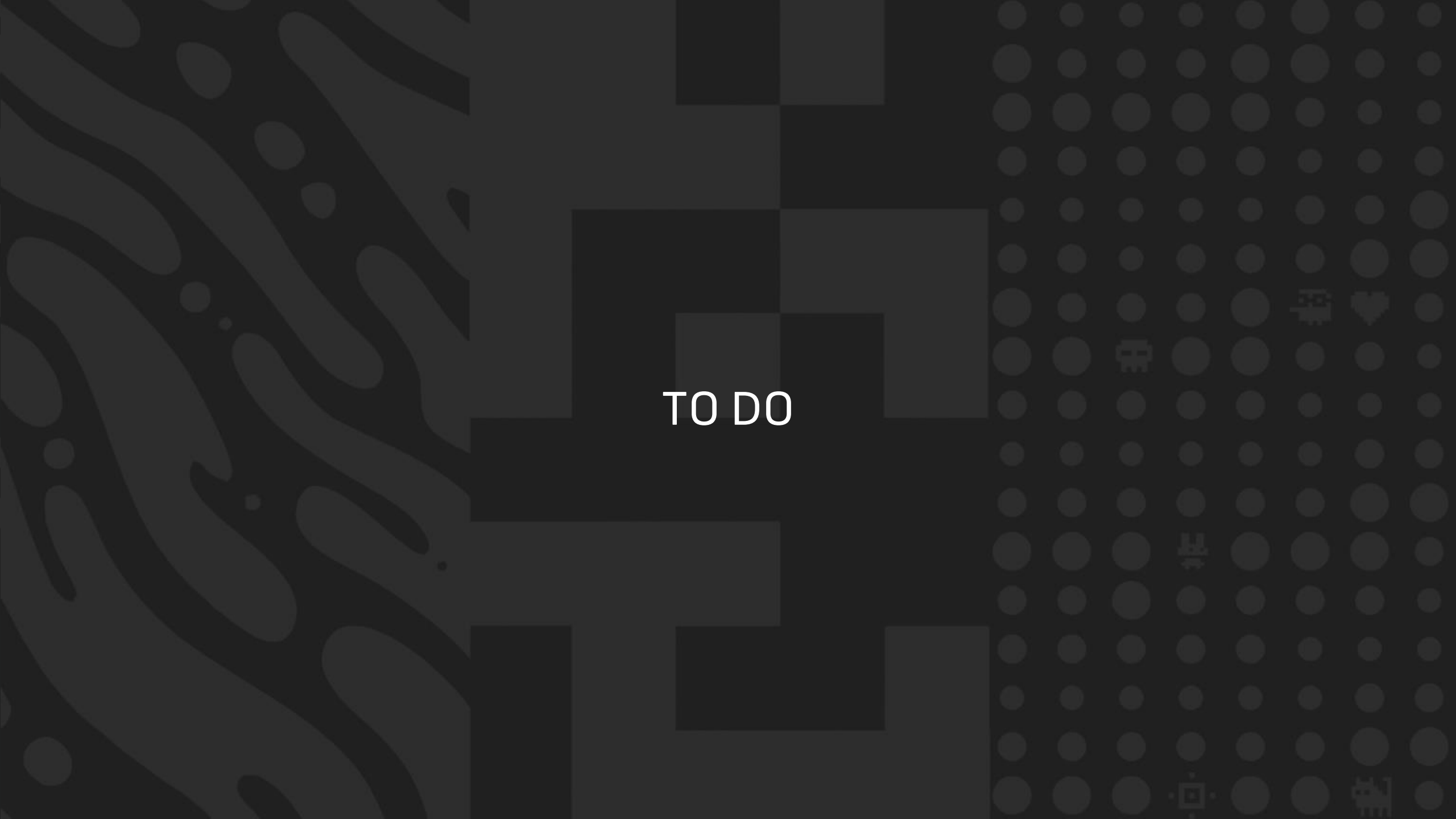
MessageType	Replication
NetID	0
Action	Update
Object Fields	type,x,y,angle
NetID	1
Action	Destroy
NetID	2
Action	Create
Fields	type,x,y,angle

ReplicationManagerClient

- Reads the replication packet
- Parses the packet and executes the replication commands (actions, update/create/destroy, etc.)
- Example:
 - ReadID
 - Read ReplicationAction
 - If ReplicationAction == Create:
 - Instantiate new object
 - Register in the linking context
 - Deserialize fields
 - Else if replicationAction == Update:
 - ...

Active/Passive Replication

- Active Replication: Each host receives the update. Each host deterministically updates the game (e.g. Lock-Step)
 - Possible issues: Fragility, hard to guarantee that states are all identical, float-point rounding, RND...
- Passive Replication: The server receives all inputs from clients. The server broadcasts all updates to each client (e.g. master-slave)
 - Possible issues: Server bandwidth/computational power, scalability



TO DO

TO DO

- Deliver a videogame with a proper World state replication system running over UDP.
- The new version of the game must have the following.
- Minimum requirements:
 - Have an explicit state replication model (active or passive)
 - Have a replication paquet that sends at least 3 types of data (e.g., position, health, actions, etc.)
 - An explicit replication manager (described in the report and implemented without bugs)
 - Accept at least 2 clients
 - Perform all the communication using UDP simulated connections.

Grading

- Grading:
- World State Replication: 60%
- Playability: 10%
- Quality of code: 10%
- Robustness and improvements from previous delivery: 20%

Grading Criteria

- World State Replication:
 - The world state is the same in both clients, with minor differences due to latency or equivalent Issues (6 points)
 - The world state is roughly the same in both clients, but there are still some issues, properly described in the report (4p)
 - The world state is not properly replicated for most of the intended parts, but the minimum requirements are covered, and I see that with a little more time, the delivery would have been better (2p)
 - The delivery is incomplete OR the state replication system is inexistent OR the game crashes (0p)

Grading Criteria

- Robustness and improvements:
 - The game is a clear improvement from the previous delivery. There are no errors.
 - There are less than 2 bugs and they are reported in the report (2p)
 - The game improves some of the previous delivery problems. There are more than 2 bugs, but they are all reported (1-1.5p)
 - The game doesn't solve any of the previous bugs or problems and/or has some unreported bugs (0-0.5p)

Grading Criteria

- Playability:
 - The game already looks like a videogame.
 - Although incomplete, it can be played and tested as a demo (1p)
 - The game is almost playable. It wouldn't be enough for a demo release, but it could be used to show off on social networks (0.5p)
 - The requirements are fulfilled, but the delivery could not be considered a videogame yet (0p)

Grading Criteria

- Code Quality:
 - The code is clean. There are no lengthy comments. There is no commented code.
 - Variables have readable names in english. Code is well structured, in multiple functions and files, each clearly organized by topic (e.g. network code, game code, others...). Indentation is consistent. (1p)
 - Between 1 and 2 of the above points is not applicable to the delivered code. (0.5p)
 - The code is difficult to understand because the above points are generally not followed. (0p)

Additional Reading

- <https://0fps.net/2014/02/10/replication-in-networked-games-overview-part-1/>
- https://www.reddit.com/r/gamedev/comments/1z0t7y/replication_in_networked_games_part_3_now_with/
- Multiplayer Game Programming (Glazer/Medhav)